viktor

The AI employee.

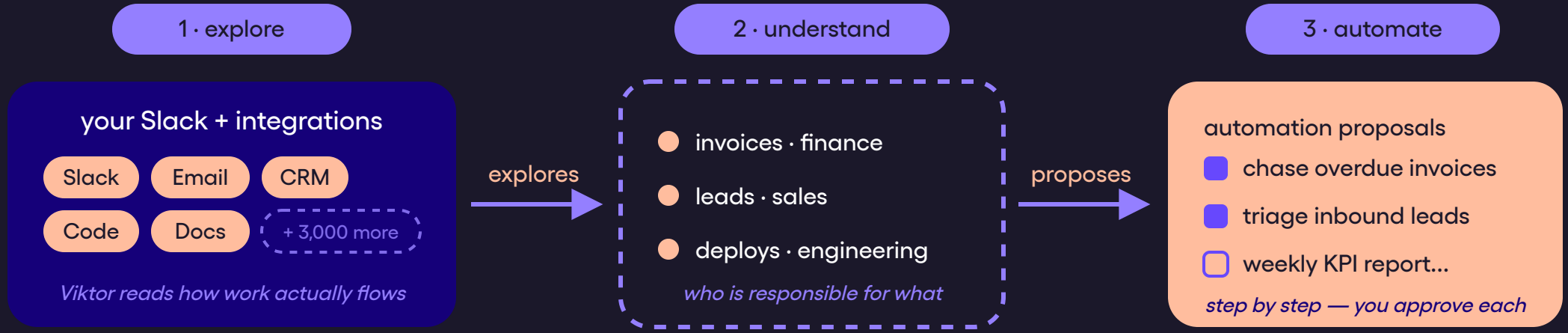
A frontier-level knowledge work agent. You talk to him in Slack or Microsoft Teams.

TUM.ai hackathon · Munich · 22–23 August 2026

Meet Viktor

● always online

- Viktor lives in Slack and Microsoft Teams. He does tasks and complex work items.
- You talk to Viktor as you talk to a teammate. He keeps context and learns over time.
- Proactive: Viktor explores your Slack and integrations, builds an understanding of who is responsible for what, and starts proposing how to automate the whole company, step by step.



Traction

and counting

55,000+

companies already run on Viktor

$75M

raised already — Viktor launched in February 2026

5M+

tool calls he makes every day — **real work, not chat**

3,200+

tools and integrations he can use out of the box

How he works

Viktor runs on his own computer in the cloud, always on. He does not follow fixed workflows. He writes and runs code.

1 Code-first harness

Every tool is a Python SDK function. Viktor writes scripts to do the work. A smarter model gets more out of the same harness.

2 Shared filesystem

Scripts and data persist across sessions. A script that worked once becomes a reusable tool.

3 Skills as memory

He stores what he learns in skill files. He loads them only when a task needs them. The tenth task is better than the first.*

4 Proactive loop

A heartbeat and scheduled tasks let him pull work like a colleague. Subagents fan out large jobs. A human approves the result.

One test for each harness decision: is it useful for the next model generation? If yes, build it.

* the rules for making this deck live in a skill file, too.

viktor

viktor

Model Router

Train a router that picks the right model for each LLM call. You set the objective.

Viktor is the AI employee in Slack and Microsoft Teams. Each of his tasks is a sequence of LLM calls.

Real agent traces, with an open off-policy problem inside.

TUM.ai hackathon · Viktor challenge track · Munich · 22–23 August 2026

The task

Input: real Viktor trajectories — each LLM call with its model, its input, and its tools. The task is open-ended. You select the objective and defend it.

A Build the router

A policy that assigns a model to each call. Optimize cost, quality, or your own trade-off.

B Prove it works

The log shows only the model that ran. Estimate what *your* router changes.

Ship: a router, an evaluation you can defend, and a short presentation.

Weekend-sized: a static offline export. Baselines train on a laptop in minutes. No GPU, no API key. The depth is in the off-policy evaluation, not in compute.

NO GPU NEEDED

The dataset

- Static offline export of real Viktor trajectories — chunked JSONL, one line for each real LLM request.
- Each line is in the OpenAI-compatible Responses format, with exactly three fields: `model`, `input`, `tools`.
- Model ids — `claude-opus-5`, `claude-fable-5`, `gpt-5.6-terra`, … (~9 ids). You infer the tier order.
- No `output`, no `usage`, no trajectory ids. We freeze the export before kickoff.

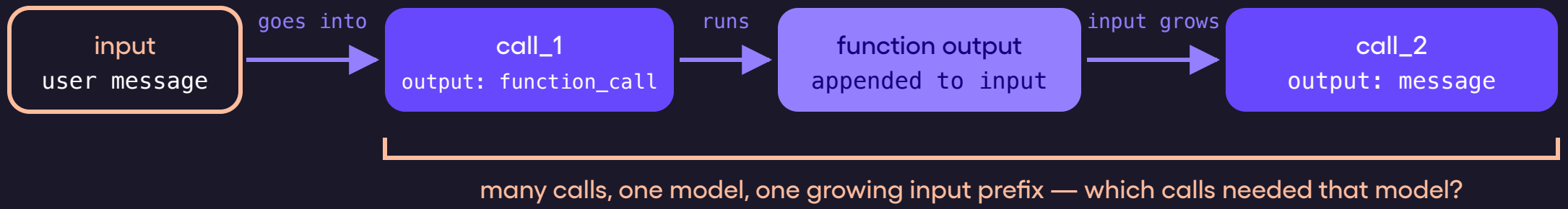
```
$ tree export/
export/
├─ trajectories_v1_01.jsonl
├─ trajectories_v1_02.jsonl
└─ ...

one line = one request.
no ids — rebuild tasks by
grouping on opening messages
```


viktor

A trajectory

- model — the id that served the call. One model serves the whole trajectory.
- input — the full request history: `message`, `function_call`, `function_call_output`, and `reasoning` items. It contains every call before it.
- tools — the function definitions the call could use: the real Viktor toolset.
- output / usage — **not shipped.** What call *i* returned appears inside call *i*+1's input — only the final output is gone. You estimate token counts yourself.



Sample

```
$ head -1 trajectories_v1_01.jsonl  # real line, abridged
{ "model": "claude-opus-5",
  "input": [
    { "role": "system", "content": "You are Viktor Ai (@viktor in slack…" },
    { "type": "message", "role": "user", "content":
        [{ "type": "input_text", "text": "…payment brief posted to [person_1] DM…" }] },
    { "type": "function_call", "name": "wait_for_background_work", "call_id": "<ID_31>", … },
    { "type": "function_call_output", "call_id": "<ID_31>", "output": "{\"wake_reason\": …}" },
    … 18 more items ],
  "tools": [ { "type": "function", "name": "bash", … }, … 12 tools ] }

// no "output". no "usage". The next request of this task repeats this
// whole input — plus what the model did. Remember this.
```

The signal

What you observe per call

- `input` items — messages, function calls, function outputs, reasoning stubs. Each call embeds every call before it.
- `tools` — the function definitions available to the call.
- What the model did — recoverable: call *i*'s output sits inside call *i*+1's input. Only each task's final output is missing.
- `model` — one id serves the full trajectory. The id varies across trajectories. **That variation is your natural experiment.**

What does not ship

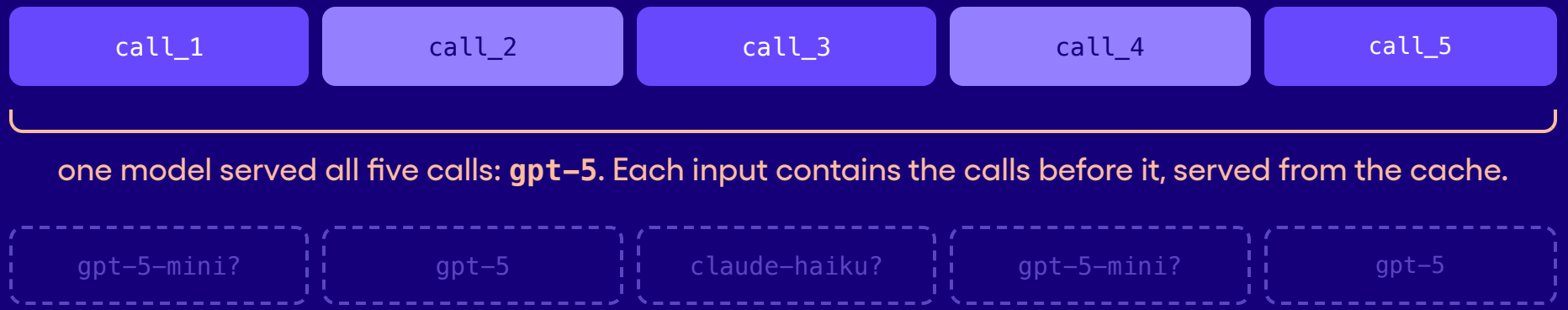
- No quality label. No timing. No `output` or `usage` fields. You estimate every token count, and you label it as an estimate.
- You construct the outcome signal yourself. Options: judge-model scores, task completion read from the final message, function-call error patterns.
- One model serves all calls of a trajectory. Which calls needed it? That question is the twist (next slide).

No routing label and no quality label ship. Constructing a good signal is part of the challenge — a good metric is a result in itself.

THE TWIST

Off-policy evaluation

ONE TRAJECTORY — FIVE CALLS



one model served all five calls: **`gpt-5`**. Each input contains the calls before it, served from the cache.

the routed alternative — *the log never shows this row. Each model switch resets the cache.*

Would a cheaper model keep the result on some calls? The log does not show it. Cost has a second trap. Providers cache the shared input prefix. A model switch resets that cache. Calls after a switch pay the full price. Estimate the two effects. The best estimate wins its own prize.

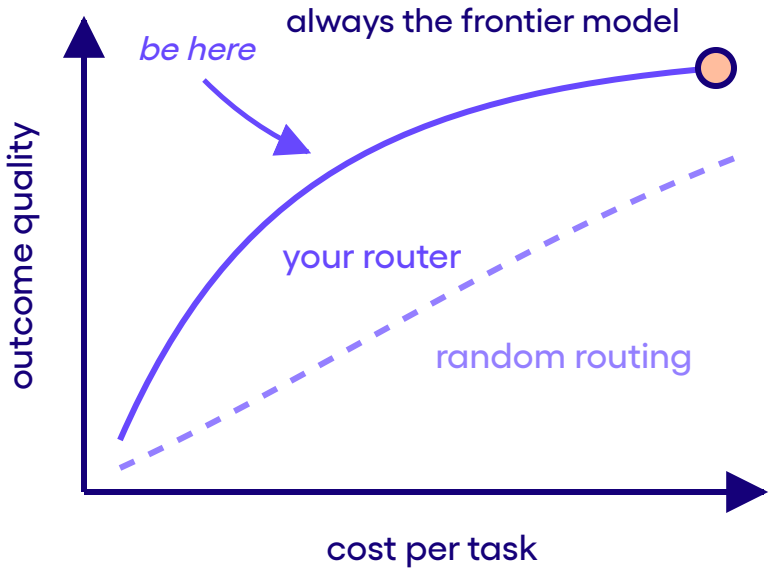
Starting points: compare calls across trajectories on different models · judge-model rescoring · cache-aware repricing from the input prefix a call shares with the one before it (no `usage` field ships).

Viktor challenge · TUM.ai

viktor

Evaluation

- Open-ended — you define the objective and the evaluation, then defend both. The held-out split ships with the dataset at kickoff.
- Cost–quality frontier — for your cost metric (tokens or dollars), show what quality your router keeps at what cost. Price it cache-aware: cached input tokens are cheap, and a model switch resets the cache. **The frontier is the headline, not a single point.**
- Off-policy honesty — the log shows one model per trajectory. Say how you estimated the change (matching, weighting, judge models). Say where the estimate can fail.
- How it is judged — three criteria, next slide.



same quality, lower cost — best frontier wins

viktor

Judging

criterion/01

Routing insight

Does your router use real structure in the traces? Can you name that structure?

What convinces the panel: a signal someone can point at, not a black box with a good score.

criterion/02

Evaluation rigor

Is the off-policy estimate honest? One model serves each trajectory, and the cache couples the calls. How did you bridge that?

What convinces the panel: you name where your estimate can fail before the panel does.

criterion/03

Presentation clarity

Can you defend your frontier in five minutes?

What convinces the panel: one chart, one claim, one known weakness.

1 build on the public split → 2 final run on the held-out split → 3 show your cost–quality frontier → 4 defend the method to the panel

We announce the criteria weights at kickoff. The special prize — best off-policy-evaluation insight — cuts across all three.

Get started

your first hour, in order

step/01 Join the challenge Discord: discord.gg/J5eNgaAT. The Viktor team answers there all weekend.

step/02 Download the starter kit. We share the link at the kickoff briefing and pin it in Discord.

step/03 Download the dataset from the kickoff link. Extract: `tar xzf trajectories_v1_01.jsonl.tar.gz -C export/`

step/04 Run `python scripts/load_trajectories.py export/`. Then run `python scripts/baseline_router.py export/`. You get reconstructed trajectories and a baseline frontier. No GPU. No API key.

step/05 Point your coding agent at the kit — `AGENTS.md` briefs it. Build your router. Beat the baseline.

Starter ideas

idea/01

the safe bet

Heuristic router

Route by prompt length and call position. One hour of work gives an honest cost floor against the logged behavior.

idea/02

the ambitious one

Learned router

Construct an outcome signal. Compare similar tasks that ran on different models. Train a classifier that picks the cheapest model that keeps the outcome.

idea/03

the special prize

Honest evaluation

Match or weight trajectories that ran on different models. Rescore single calls with a judge model.

idea/04

check before you celebrate

Cache-aware costing

Reprice each route with cached and uncached token rates — infer each call's cached share from the input prefix it shares with the previous call. Show how much a naive estimate overstates your savings.

Rules & prizes

Rules & logistics

`rule/01 · format`

Team size, submission format, and deadline follow the general TUM.ai hackathon rules. We confirm them at the kickoff briefing.

`rule/02 · compute`

We announce compute and API credits for the track at kickoff. The dataset runs on a laptop.

`rule/03 · license`

One line: do not use the data outside this challenge. Do not redistribute it.

Either way you leave with a portfolio piece — model routing and off-policy evaluation on real agent traces — and hiring conversations: viktor.com/jobs.

Prizes — on top of the base TUM.ai prizes

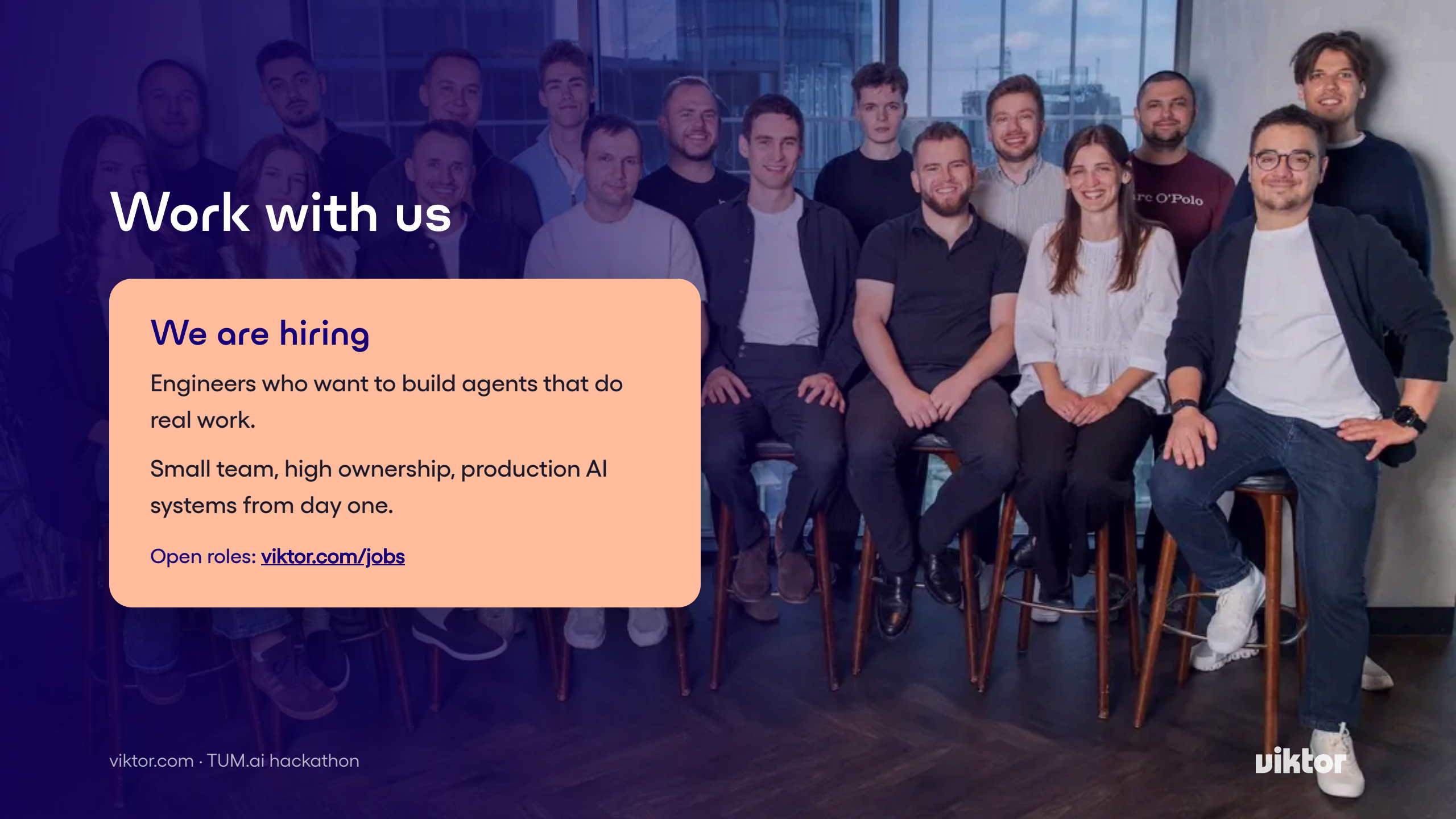
+ a day trip to the Warsaw office



Special prize for the best off-policy-evaluation insight — slide 15, card 03 is the map.

viktor

Work with us



We are hiring

Engineers who want to build agents that do real work.

Small team, high ownership, production AI systems from day one.

Open roles: viktor.com/jobs

viktor

Contact

One dataset, one weekend, one question: which model should have taken this call?

- Mentors: **Bojan** and **Peter** are on site all weekend. **Ivan** mentors remotely. We announce the table location and office hours at kickoff.
- Challenge Discord: discord.gg/J5eNgaAT — challenge assets, direct access to the Viktor team, and a pre-configured Viktor instance to try.
- Dataset download link: we share it in the kickoff briefing.
- Careers: viktor.com/jobs

P.S. Viktor built this deck himself. He is curious what you build.

P.P.S. He left one more message in the browser console.

Challenge Discord —
discord.gg/J5eNgaAT
Viktor is already in there.

viktor.com — who we are.
Brief + starter repo link: at kickoff

viktor